FEDERATED LEARNING FOR CODE GENERATION WITH HALLUCINATION – AWARE FINE TUNING

A PROJECT REPORT

Submitted by

ABHINAVH P

PRABHAKARA ARJUN R

BUVANES SRIVARDAN K

COURSE CODE: CS6811

COURSE TITLE: PROJECT WORK

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600 044

APRIL 2026

DEPARTMENT OF COMPUTER TECHNOLOGY

ANNA UNIVERSITY, MIT CAMPUS

CHROMEPET, CHENNAI – 600 044

BONAFIDE CERTIFICATE

Certified that this project report **"FEDERATED LEARNING FOR CODE GENERATION WITH HALLUCINATION – AWARE FINE TUNING"** is the bonafide work of **Mr. Abhinavh P (2022503059), Mr. Prabhakara Arjun R (2022503003) & Mr. Buvanes Srivardan K (2022503037)** who carried out the project work under my supervision.

SIGNATURE OF HOD

Dr. PONSY R K SATHIA BHAMA
Head of Department
Department of Computer Technology
Anna University, MIT Campus
Chromepet, Chennai-600044

SIGNATURE OF SUPERVISOR

Dr. THANASEKHAR B
Professor
Department of Computer Technology
Anna University, MIT Campus
Chromepet, Chennai-600044

ABSTRACT

Large Language Models (LLMs) have demonstrated strong capabilities in automated code generation, enabling solutions for a wide range of software engineering tasks. However, these models often suffer from hallucinations, producing syntactically correct but semantically incorrect code, including invalid APIs and logical errors. Such issues reduce the reliability of LLM-based systems in practical applications.

In this project, we propose a federated learning-based framework for improving code generation by reducing hallucinations in a privacy-preserving manner. The system integrates hallucination detection to identify and correct errors in generated code, thereby creating high-quality datasets for model adaptation. These datasets are used to fine-tune the model using parameter-efficient Low-Rank Adaptation (LoRA) across multiple decentralized clients.

Each client performs local fine-tuning using its own data, and only lightweight adapter parameters are shared with a central server, ensuring data privacy. Unlike traditional federated learning approaches that use standard Federated Averaging (FedAvg), this work introduces a novel aggregation strategy called Fed-Error Average, where model updates are weighted based on both dataset size and the distribution of hallucination errors across clients.

This error-aware aggregation mechanism enables the model to learn more effectively from diverse error patterns, leading to improved generalization and reduced hallucinations. Experimental results demonstrate that the proposed approach achieves better correctness compared to baseline models.

ACKNOWLEDGEMENT

We take this humble opportunity to thank Dean, MIT Campus, Anna University, **Dr. P Jayashree and Dr. Ponsy R K Sathia Bhama**, Associate Professor & Head, Department of Computer Technology, MIT Campus, Anna University for providing all the lab facilities in pursuit of this project.

Undertaking this project has helped us learn a lot, and we would like to express our sincere gratitude towards our supervisor **Dr. B.Thanasekhar**, Professor, Department of Computer Technology, MIT, Anna University, whose kind guidance, and proper directions helped shape this project perfectly. The timely feedback from the supervisor was very instrumental in successful completion of the project work carried out. Constant encouragement to implement the project to the fullest of our potential led us through, during the difficult phase of the project, and assisted in the successful completion of the proposed project work.

We acknowledge the efforts and feedback of the panel members, **Dr. P. Pabitha**, Associate Professor, Department of Computer Technology, MIT, Anna University, **Dr. S. Chithra**, Associate Professor, Department of Computer Technology, MIT, Anna University and **Dr. V.P. Jayachitra**, Assistant Professor, Department of Computer Technology, MIT, Anna University in reviewing our work, providing constant valuable comments and encouraging us to view the different aspects of the project in successful implementation of the project.

We thank all the teaching and non-teaching members of the Department of Computer Technology, MIT, Anna University, seniors, juniors, and friends whose appreciable help, either directly or indirectly helped in the smooth completion of the Creative and Innovative Project in a limited time frame.

ABHINAVH P

PRABHAKARA ARJUN R

BUVANES SRIVARDAN K

BLES viii

CHAPTER NO. LIST OF 31

Formatted: Right: 0.52 cm

Formatted: Heading 1, Indent: Left: 0 cm

TABLE OF CONTENTS

TITLE PAGE NO. LIST OF TA

Formatted: Indent: Left: 0 cm

~~LIST OF TABLES~~

Formatted: Comment Subject1, Left, Right: 0 cm, Space Before: 0 pt

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	SUMMARY OF LITERATURE SURVEY	15

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Client-Side hallucination Detection And Dataset creation	11
3.2	LoRA Fine-Tuning and Federated Aggregation	13
3.3	Fed-Error Avg	19
4.1	Schema for AST module	29
4.2	Schema for dynamic analysis module	30
4.3	Schema for LIB_API module	31
4.4	Dataset Producing its own set of LoRA Adaptors	33
4.5	Output of fine-tuned adaptors after Federated average (Complete dataset)	35
4.6	Output of fine-tuned adaptors after Federated error average (Complete dataset)	35
4.7	Output of fine-tuned adaptors after Federated average (Training dataset)	35
4.8	Output of fine-tuned adaptors after Federated error average (Training dataset)	35
4.9	Model and Tokenizer loaded	36
5.1	Hallucination and Passed Count by Dataset	36
5.2	Baseline Vs FedAvg Error Count by Dataset (Train)	38
5.3	Baseline Vs FedAvg Error Count by Dataset	38

5.4	Baseline Vs FedErrorAvg Error Count by Dataset (Train)	3~~2~~39
5.5	Baseline Vs FedErrorAvg Error Count by Dataset	~~2~~339
5.6	Baseline Vs FedAvg Vs FedErrorAvg Error Pass Rate by Dataset (Train)	40
5.7	Baseline Vs FedAvg Vs FedErrorAvg Error Pass Rate by Dataset (TT)	41
5.8	CodeBLEU : Baseline Vs FedAvg Vs FedErrorAvg by Dataset (Train)	42

CHAPTER 1

INTRODUCTION

1.1 LARGE LANGUAGE MODELS IN CODE GENERATION

The emergence of large language models (LLMs) represents a major milestone in artificial intelligence and software engineering. These models, primarily based on transformer architectures, are trained on massive corpora consisting of source code, technical documentation, and natural language text. As a result, they can generate syntactically valid and semantically coherent code across multiple programming languages and domains.

Code generation-the automatic synthesis of executable programs from natural language or partial code-has long been a goal in software engineering. Earlier approaches, including rule-based systems, template-driven methods, and sequence-to-sequence models, were limited in scope, struggled with ambiguity, and lacked cross-domain generalization. LLMs overcome these limitations by leveraging large-scale pretraining, enabling transferability across tasks, languages, and abstraction levels.

Modern LLM-based systems such as GitHub Copilot, Codex, AlphaCode, and Code Llama demonstrate strong capabilities in tasks like function completion, test generation, code translation, and even multi-file program synthesis. Their widespread adoption in industry highlights both their practical utility and the importance of understanding their limitations.

Despite these advances, LLM-based code generation is not fully reliable. A key issue is hallucination-the generation of code that appears correct but is logically flawed, non-functional, or based on non-existent APIs. These errors are often subtle and can have serious consequences in real-world software systems. Therefore, understanding and detecting hallucinations in LLM-generated code

remains a critical area for research.

1.2 HALLUCINATIONS IN CODE GENERATIONS

Despite the advancements in Large Language Models (LLMs) for code generation, ensuring the correctness and reliability of generated code remains a significant challenge. One of the major issues is hallucination, where the model produces syntactically correct but semantically incorrect code, including invalid API usage, logical inconsistencies, and runtime failures.

Existing approaches such as prompt engineering, repeated sampling, supervised fine-tuning, and reinforcement learning from human feedback (RLHF) provide partial improvements. However, these methods do not guarantee program correctness and often fail to address deeper structural and execution-level errors.

Another key challenge arises in real-world environments where data required for improving model performance is distributed across multiple clients. Due to privacy and data ownership concerns, this data cannot be centrally collected for training. Traditional centralized learning approaches are therefore not suitable for such scenarios.

Furthermore, standard federated learning methods, such as Federated Averaging (FedAvg), aggregate model updates based only on dataset size and do not consider the variation in error patterns across clients. This limits the model's ability to effectively learn from diverse hallucination distributions.

These challenges highlight the need for a framework that ensures reliable code generation while enabling decentralized, privacy-preserving learning with improved aggregation strategies.

1.3 FEDERATED LEARNING

The limitations of existing code generation approaches highlight the need for a more robust and scalable solution that ensures both correctness and adaptability. While Large Language Models (LLMs) are capable of generating code efficiently, their outputs often lack reliability due to hallucinations and insufficient validation mechanisms.

At the same time, real-world data required to improve model performance is distributed across multiple clients and cannot be centrally aggregated due to privacy and data ownership constraints. This creates a need for decentralized learning approaches that can utilize distributed data without compromising privacy.

To address these challenges, a system is required that not only detects and corrects hallucinations but also enables collaborative learning across multiple clients. Integrating hallucination detection and automatic program repair helps improve the quality of generated code and creates reliable training data.

Furthermore, traditional federated learning approaches are not sufficient, as they do not consider the variation in error patterns across different clients. Hence, there is a need for an improved aggregation strategy that can effectively utilize error information during model updates.

The proposed system addresses these requirements by combining hallucination-aware processing with a federated learning framework and introducing an error-aware aggregation method to enhance overall model performance

1.4 PARAMETER EFFICENT FINE TUNING

The scale of modern large language models-ranging from millions to hundreds of billions of parameters-makes full fine-tuning computationally expensive and often impractical without large GPU infrastructure. It requires updating all parameters, storing gradients, and maintaining optimizer states, leading to significant memory and compute overhead. Parameter-efficient fine-tuning (PEFT) addresses this by adapting a mostly frozen pre-trained model using a small set of trainable parameters, significantly reducing resource requirements.

Among PEFT methods, Low-Rank Adaptation (LoRA) is widely used, representing weight updates as the product of two low-rank matrices, thereby reducing trainable parameters. Other approaches include prefix/prompt tuning, which inject trainable vectors into inputs or hidden states, and adapter modules, which add small bottleneck layers within transformer blocks. These methods trade slight reductions in flexibility for major gains in efficiency.

For code-generating LLMs, PEFT is particularly valuable in reducing hallucinations, as it enables targeted fine-tuning on curated datasets without overwriting general knowledge. This helps improve API correctness and generation reliability while preserving overall capability. When combined with federated learning, PEFT further reduces communication costs, since only lightweight adapter parameters need to be shared instead of full model updates.

1.5 PROBLEM STATEMENT

Large Language Models (LLMs) used for code generation often produce hallucinated outputs, resulting in syntactically correct but semantically incorrect code. These inaccuracies reduce the reliability of such models in real-world software development scenarios. While existing approaches attempt to

improve performance through fine-tuning and prompt engineering, they do not effectively leverage distributed data available across multiple clients.

In practical environments, data required for improving model performance is decentralized and cannot be shared due to privacy and ownership constraints. Although federated learning provides a solution for decentralized training, traditional aggregation methods such as Federated Averaging (FedAvg) rely only on dataset size and fail to consider variations in error patterns across clients. This limits the model's ability to learn effectively from diverse hallucination distributions.

Therefore, there is a need for a privacy-preserving framework that not only enables collaborative learning across distributed clients but also incorporates error-aware aggregation strategies to improve the reliability and correctness of LLM-based code generation.

1.6 OBJECTIVE

The objective of this project is to develop a federated learning-based framework to improve the reliability of large language model (LLM)-based code generation, with a primary focus on parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA). The framework aims to enable collaborative model training across multiple decentralized clients, allowing local fine-tuning on private codebases while preserving data privacy and reducing computational and communication overhead.

Another objective is to design and integrate a hallucination-aware pipeline that detects and classifies errors in generated code using both static and dynamic program analysis techniques. This includes identifying syntactic, semantic, API-related, and logical hallucinations, and establishing a structured taxonomy to

support consistent evaluation and targeted model refinement.

The project further aims to develop an error-aware aggregation strategy, termed Fed-Error Average, which incorporates the distribution of hallucination errors from different clients during federated aggregation. This is intended to improve learning from heterogeneous and non-IID data distributions and enhance the robustness of the global model.

Finally, the objective is to evaluate the effectiveness of the proposed framework using benchmark datasets by measuring improvements in functional correctness (e.g., pass@k) and reductions in hallucination frequency and severity, thereby establishing a privacy-preserving and error-aware fine-tuning paradigm for reliable LLM-based code generation.

CHAPTER 2

LITERATURE SURVEY

Yang et al., 2025 [1] proposed a hybrid approach for hallucination detection in LLM-generated code by combining static and dynamic analysis techniques. The system analyzes code using Abstract Syntax Trees (AST) and validates execution through runtime testing to detect both structural and functional errors. The approach improves detection accuracy across multiple error types. However, it depends heavily on test case coverage and may fail to detect deeper semantic inconsistencies. Additionally, scalability in distributed environments remains a challenge.

Fakhoury et al., 2024 [2] proposed a test-driven interactive code generation framework that integrates LLM outputs with user-defined test cases. The system improves code reliability by validating generated outputs against test cases during generation. However, the approach relies on the availability of high-quality test cases and may not perform well when test coverage is incomplete. It also lacks mechanisms for correcting detected errors automatically.

Chen et al., 2025 [3] proposed a federated adaptive fine-tuning framework for Large Language Models in software development. The system enables decentralized training across multiple clients while preserving data privacy by sharing only model updates. Although the framework improves adaptability across distributed datasets, it does not incorporate error-aware learning or hallucination-specific optimization. Aggregation is still based on standard federated averaging, limiting its effectiveness in heterogeneous environments.

Nashaat and Miller, 2024 [4] proposed an efficient fine-tuning approach using organizational data to improve software-related tasks. The system reduces

computational overhead by using parameter-efficient fine-tuning techniques. However, the approach is designed for centralized environments and does not consider distributed data scenarios. It also does not address hallucination issues in generated code.

Khojah et al., 2025 [5] analyzed the impact of prompt programming on function-level code generation. Their study shows that structured prompts improve code quality and reduce minor errors. However, prompt engineering is highly sensitive and does not eliminate deeper logical or semantic errors. The approach lacks robustness across diverse tasks and does not address hallucination systematically.

Wang et al., 2025 [6] proposed an adaptive framework integrating LLMs for knowledge graph construction. The system improves structured knowledge extraction using LLM capabilities. However, the framework is not specifically designed for code generation tasks and does not address hallucination in programming contexts. Its applicability to software engineering problems remains limited.

Yu et al., 2024 [7] evaluated the reliability of LLMs in source code-related tasks and demonstrated that model performance varies significantly across datasets and task complexity. The study highlights the presence of hallucinations and inconsistencies in generated outputs. However, it does not propose a concrete solution for mitigating these issues and lacks integration with learning frameworks.

Yi et al., 2025 [8] proposed FedALoRA, which combines Low-Rank Adaptation (LoRA) with federated learning to enable efficient and privacy-preserving model training. The system allows local fine-tuning using lightweight adapters and reduces communication overhead. However, the approach focuses on

general NLP tasks and does not consider hallucination-specific optimization in code generation.

Liu et al., 2024 [9] assessed the quality of code generated by LLMs and found that while outputs are often syntactically correct, they frequently fail functionally due to logical errors and incorrect assumptions. The study highlights limitations in reasoning capabilities of current models. However, it does not provide mechanisms for correcting hallucinated outputs.

Ouyang et al., 2025 [10] proposed a knowledge-enhanced program repair approach that uses contextual information to guide patch generation. The system improves repair accuracy compared to traditional methods. However, it depends on the availability of structured knowledge and may struggle with complex or unseen error patterns.

Liu et al., 2025 [11] proposed a method to address API hallucinations by injecting external knowledge into LLMs. The approach improves correctness by guiding the model with verified API information. However, it is limited to API-level errors and does not address broader logical or structural hallucinations.

Zhang et al., 2025 [12] proposed EvoAPR, which combines genetic algorithms with LLM-based program repair and dynamic LoRA adaptation. The system iteratively improves patch quality and repair success rates. However, it is computationally expensive and may not scale efficiently for large datasets or real-time applications.

CHAPTER 3

PROPOSED WORK

3.1 SYSTEM OVERVIEW

Formatted: List Paragraph

The proposed system is organized into two tiers: the client tier and the server tier. These tiers interact through parameter exchange mechanism, where only LoRA adapter weights are shared, never the raw data.

At the client tier, each participating client operates as a fully independent code generation and fine-tuning unit. A client receives a benchmark dataset or dataset partition, generates code using the frozen base LLM then subjects the generated code to a hallucination detection pipeline, and constructs a local training dataset from the outcomes. The client then performs supervised fine-tuning(SFT) using LoRA on this locally assembled dataset, producing a set of client-specific adapter weights. These weights encode the learning the model derived from that client's error patterns and corrected programs.

The server tier consists of a central aggregation server whose sole responsibility is to collect LoRA adapters from all clients at the end of each communication round, perform weighted averaging to produce a global adapter, and redistribute that global adapter back to all clients. The server performs no code generation or evaluation; it is purely a coordinator. It is important to note that it never has access to the raw datasets, generated code, or problem statements from any client. The full pipeline is illustrated in Figure 3.1 and Figure 3.2.

3.2 METHODOLOGY WITH ARCHITECTURE DIAGRAM

3.2.1 Phase 1 : Client-Side Hallucination Detection and Dataset Construction

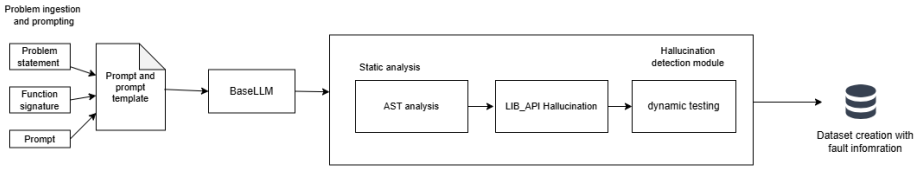


Figure: 3.1 – Client-side hallucination detection and dataset construction framework

The first phase is executed entirely within each client and encompasses all activities from code generation through dataset preparation. Refer to Figure 3.1 ,It begins with problem ingestion, where benchmark datasets (HumanEval, MBPP, and DS-1000) are loaded and each client is assigned a distinct dataset or partition to simulate heterogeneous data distributions in a real federated environment. For each task, a structured prompt is constructed by combining the problem description and the function signature. This prompt is then fed to the frozen Qwen2.5-Coder-3B model using single-shot prompting, producing an initial code solution.

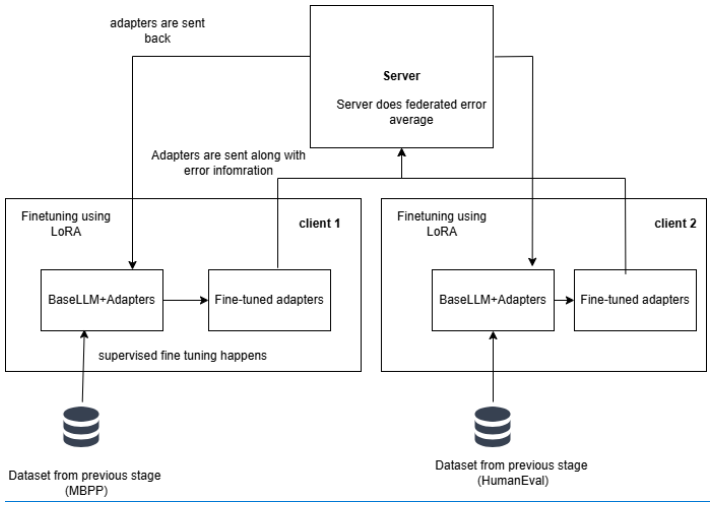
Once code is generated, it enters the hallucination detection pipeline. Static analysis is performed first using Python's AST module, which parses the generated code and inspects its structural representation. Following static analysis, a library and API validation step checks whether the functions, methods, and attributes called in the code correctly correspond to the actual interfaces provided by the libraries available in the execution environment.

After static analysis, the code undergoes dynamic analysis by executing it against the test cases provided by the benchmark dataset. Execution takes place in a controlled environment with timeout protection to prevent infinite loops from stalling the pipeline. Any runtime exceptions - including AssertionError, NameError, TypeError, ValueError, IndexError, and others - are captured along with their traceback information, including error type, error message, and the line number where the failure occurred. Failures in assertion checks, which indicate semantic or logical errors, are also recorded alongside the expected and actual outputs for each test case.

These fault records, together with the original prompt, the generated code, and the canonical solution from the dataset, form the raw material for dataset construction.

Dataset construction organizes the above information into a structured format with columns including the dataset name, task identifier, prompt, status, AST analysis information, dynamic analysis information, library analysis information, the generated code, the target code (canonical solution), error sources, error types, error lines. The prompt and canonical solution columns are the primary training signal used in the subsequent LoRA fine-tuning stage. The error type and frequency information is recorded separately to be reported to the server for use in the Fed-Error Average aggregation.

3.2.2 Phase 2 : LoRA Fine-Tuning and Federated Aggregation



~~<diagram>~~*Figure: 3.2 LoRA Fine tuning and federated aggregation*

Refer to Figure 3.2,the second phase begins once the client has assembled its dataset from Phase 1. The base Qwen2.5-Coder-3B model is loaded and base model is frozen and LoRA adapter modules are injected into all seven projection matrices of each transformer layer.This comprehensive injection ensures that the adapter can influence both the attention and feed-forward computations across all 36 transformer layers of the model(QwenCoder).

The LoRA rank is set to 16 and the scaling factor alpha to 32, yielding an effective scaling multiplier of 2. With these setting we get 504 adapter in total, corresponding to 36 layers. The total number of trainable LoRA parameters across the model is approximately 29.9 million, which is a small fraction of the full 3.09 billion parameters. All original model weights remain strictly frozen throughout training; only the LoRA adapter matrices are updated.

Training is conducted using supervised fine-tuning (SFT) . After training completes, the LoRA adapter weights are saved as a .pt checkpoint file and prepared for transmission to the central server. No model weights, no raw data, and no code examples leave the client at this stage; only the compact adapter checkpoint is sent.

At the server, the Fed-Error Average aggregation algorithm is applied. First, the server collects the adapter checkpoints and the error frequency reports from all participating clients. For each error type e observed across all clients, the total error frequency F_e is computed by summing the counts of that error type across all clients. A normalized weight is then assigned to each error type based on its global frequency, reflecting how prevalent that category of error is across the federated system. For each client k, an error score E_k is computed by multiplying the count of each error type on that client by the corresponding global weight and summing across all error types. This score quantifies how rich or diverse the client's error distribution is relative to the overall population.

Each client's aggregation weight α_k is obtained by normalizing its error score E_k against the sum of all clients' error scores. These weights replace the sample-size-based weights used in standard FedAvg. The global adapter at round t+1 is then computed as the weighted average of all clients' adapter weights at round t, with each client weighted by its α_k. The resulting global adapter is redistributed to all clients, who load it alongside the frozen base model to obtain an improved code generation system. This completes one federated communication round, and the process can be repeated for subsequent rounds.

3.3 MODULES OF THE PROPOSED SYSTEM

3.3.1 Code Generation Module

This module is responsible for loading benchmark datasets and generating initial code solutions from the frozen base language model. Here, benchmark code generation datasets such as MBPP, HumanEval, and DS-1000 are loaded from Hugging Face datasets library. Code generation is done by single-shot prompting .

The frozen Qwen2.5-Coder-3B pretrained model is used to generate the initial code solutions . Code generation is performed with deterministic settings (do_sample=False, temperature=1.0) to ensure reproducibility. The output of this module for each task is a raw generated code string, which may or may not contain hallucinations, and which serves as the input to the hallucination detection module.

In the federated setting, different clients are assigned different datasets. Client 1 operates on MBPP, while Client 2 operates on HumanEval, Client 3 operates with DS1000 simulating the heterogeneous data distributions that would arise when different organizations generate code for different domains.

3.3.2 Hallucination Detection Module

The hallucination detection module constitutes the analytical core of the client-side pipeline. It accepts a generated code string as input and produces a structured fault record as output. The module operates in two sequential stages: static analysis and dynamic analysis.

The AST module is the first stage of hallucination detection and inspects code without executing it. It parses the Python source into a tree. If parsing fails due to invalid syntax or indentation, it records the error, its line number, and

stops, since the code cannot be executed. When parsing succeeds, it traverses the tree and checks that control-flow constructs are used correctly: return only inside functions, and break and continue only inside loops; otherwise it reports structural violations. It also performs flow-aware name analysis: it follows control flow through conditionals, loops, try-except blocks, and nested scopes, and tracks where variables are defined and used.

It reports uses of variables that are never defined on any path, including in branches that tests may not run. It can therefore detect syntax errors, indentation errors, structural misuses, and undefined variables purely from the code structure, without running tests. Because it works statically, it can find these defects even in rarely executed paths, unlike dynamic testing, which only detects errors on paths actually covered by tests. Any reported error causes later stages (dynamic execution and API checks) to be skipped, because the code is not valid for execution.

The LIB_API module performs static checks on how external libraries and APIs are used in the generated code. It does not run the code; it inspects the source to validate imports, attribute access, and function calls. For imports, it checks that imported modules exist and can be loaded. For attribute access (e.g., obj.attr), it checks that the attribute exists on the object. For function and method calls, it checks that the number and types of arguments match the expected signature. It reports missing modules, invalid attributes, and invalid arguments. The module distinguishes between failures caused by missing libraries in the environment and real API misuse in the generated code, so only actual hallucinations are flagged. Its output includes counts of module-not-found, attribute, type, and name errors, object, attribute, or function involved and the line number, which helps locate and fix library-related hallucinations.

The dynamic analysis module executes the generated code with the benchmark's test cases and observes what happens at runtime. It runs the code in

an isolated environment with a timeout to avoid hanging on infinite loops. If execution completes and all tests pass, the code is marked as passed. If execution raises an exception or a test fails, it records the status as failed along with the exception type, message, and the line number where it occurred. It can report runtime errors such as type mismatches, attribute or name errors, key errors, assertion failures (wrong output), and timeouts (too slow or non-terminating). For failing tests, it records the failing input, the expected output, and the actual output. This module only sees defects that occur on paths executed by the test cases; it cannot detect errors in code paths that are never run, which is why it is complemented by static analysis modules such as AST and LIB_API that check structure and API usage without running the code.

3.3.3 Dataset Creation

The dataset construction module transforms the raw outputs of the hallucination detection module into a structured training dataset suitable for LoRA fine-tuning. For each task processed by the client, the module assembles a record containing the dataset name, task identifier, prompt, hallucination status, AST analysis information, dynamic analysis information, library analysis information, the generated code, canonical target code, the error sources, the error types, the error lines, and the canonical solution from the benchmark. An automatic program repair (APR) module was initially intended to correct hallucinated code using the detected error information and patched code; however, in practice it often failed to produce valid fixes even when given precise fault locations and error types, owing to limitations of the Qwen coder model in reliably repairing code from such inputs.

Given this, the system does not rely on APR outputs as training targets. Instead, programs that passed all hallucination checks are stored with the generated code as the target, while programs that failed detection are stored with the canonical solution from the benchmark as the target, since the model's

generation is unreliable in those cases and APR could not reliably fix it. The prompt and canonical solution columns serve as the input–output pairs for supervised fine-tuning. This design ensures that fine-tuning always uses a correct reference target, even when the model's own generation was hallucinated.

The main aim is to achieve hallucination reduction in a federated setting rather than through detection and repair; the hallucination framework is used to assess code correctness via pass@1 evaluation.

3.3.4 LoRA Fine-Tuning Module

The LoRA fine-tuning module performs parameter-efficient supervised fine-tuning of the base model using the dataset assembled by the dataset construction module. Low-Rank Adaptation (LoRA) decomposes weight updates into products of two low-rank matrices, drastically reducing the number of trainable parameters while preserving the expressive capacity needed to adapt to new data patterns.

LoRA modules are injected into all seven projection matrices of each of the 36 transformer layers in the base model: the query (q_proj), key (k_proj), value (v_proj), and output (o_proj) projections in the self-attention sublayer, and the gate ($gate\_proj$), up (up_proj), and down ($down\_proj$) projections in the MLP sublayer.

The base model weights remain fully frozen throughout training, and only the LoRA adapter matrices are updated via gradient descent. An effective batch size of 8 is achieved through gradient accumulation (per-device batch size of 1, accumulation steps of 8), and training runs for 3 epochs over the local dataset. At the end of training, only the LoRA adapter weights , not the full model are saved to disk as a .pt checkpoint file. This tensors is sent back to servers.

3.3.5 Federated Aggregation Module

The federated aggregation module operates on the central server and is responsible for combining the LoRA adapter received from all clients into a single global adapter. The module implements the proposed Fed-Error Average algorithm, which extends standard Federated Averaging by incorporating hallucination error frequency information alongside dataset size.

Algorithm 1 Fed-Error Average: Frequency-Weighted Federated Aggregation

Require: Client LoRA adapters $\{w_k^t\}_{k=1}^K$, error counts $\{\text{count}_{k,e}\}$
Ensure: Global adapter w^{t+1}

Step 1: Compute error frequency for each error type e
    $F_e \leftarrow \sum_{k=1}^{K} \text{count}_{k,e}$
Step 2: Compute error-type weight
    $\text{weight}_e \leftarrow \frac{F_e}{\sum_{e' \in \mathcal{E}} F_{e'}}$
Step 3: Compute client error score E_k
    $E_k \leftarrow \sum_{e \in \mathcal{E}} \text{count}_{k,e} \cdot \text{weight}_e$
Step 4: Normalize client weights
    $\alpha_k \leftarrow \frac{E_k}{\sum_{j=1}^{K} E_j}$
Step 5: Aggregate client adapters
    $w^{t+1} \leftarrow \sum_{k=1}^{K} \alpha_k \, w_k^t$
return w^{t+1}

Fig 3.3 : Algorithm for Federated error Average

The aggregation process begins by computing, for each error type e, the global error frequency F_e as the sum of counts of that error type across all clients. A normalized weight for each error type is then derived by dividing F_e by the total count of all error types observed globally. For each client k, an error score E_k is computed as the weighted sum of the client's per-type error counts, using the global error type weights. This score reflects not just how much data the client has, but how diverse and significant its error distribution is relative to the global picture. The client's aggregation coefficient α_k is obtained by normalizing E_k against the sum of all clients' error scores.

The global adapter at round t+1 is computed as the sum over all clients of $\alpha_k$times the client's local adapter weights w_k^t. This weighted average is applied element-wise across all 504 adapter matrices. The resulting global adapter is then send back to all clients. Each client merges the global adapter with the frozen base model to obtain the updated code generation system for inference or for further rounds of local fine-tuning.

The advantage of this formulation over standard FedAvg is that clients whose local datasets reflect more complex or more diverse hallucination patterns for instance, a client that observed many distinct error types versus one that saw mostly a single error type contribute proportionally more to the global update. This is particularly important in heterogeneous federated settings, where different clients encounter qualitatively different failure modes in code generation.

3.3.6 Verification Module

The verification module checks whether the fine-tuned LoRA adapters improve code generation. It loads the base model plus the LoRA adapter (FedAvg or Fed-Error-Avg), generates code for each benchmark task, and runs the full hallucination pipeline (AST, dynamic execution, and LIB_API) on that output. The result is a pass-or-fail status for each task, recorded in a CSV with the same format as the baseline. By comparing baseline (no adapter) and adapter runs, it measures how many more tasks pass after fine-tuning and whether the federated adapters reduce hallucinations and improve Pass@1.

3.3.7 Formualas Used

1. Basic LoRA

LoRA reduces the number of trainable parameters by decomposing the weight update into two low-rank matrices, allowing efficient fine-tuning without modifying the full model weights. Below is the mathematical split-up and working of it.

Let W_0 be the weight of the base model,

$$W_0 \in R^{d \times k}$$

Now we can breakdown this W_0 into two adapters B and A (LoRA matrices) with a rank r,

$$\Delta W = BA$$

Thus B and A are,

$$B \in R^{d \times r}, \quad A \in R^{r \times k}$$

Updated weights,

$$W = W_0 + BA$$

Scaled LoRA used at experiment,

$$W = W_0 + \frac{\alpha}{r} BA$$

r – rank of low-rank decomposition
α – scaling factor

2. Federated average

Standard federated averaging aggregates local model weights from all clients into a single global model by taking a uniform weighted mean across participants. This is the conventional method.

$$w^{t+1} = \sum_{k=1}^{K} \frac{n_k}{N} w_k^t$$

K - client

n_k = number of training samples on client k

N – total number of samples used across all clients

3. Augmented Fed-average(Frequency-weighted/Fed-ErrorAverage)

This extends standard federated averaging by assigning higher aggregation weights to clients that encountered more errors during training, ensuring the global model learns more from clients where hallucinations were harder to resolve.

Error frequency ,

$$F_e = \sum_{k=1}^{K} count_{k,e}$$

F_e = total occurrences of error type 'e' across all clients
$count_{k,e}$ = number of errors of type 'e' in client k

Weight Definition,

$$weight_e = \frac{F_e}{\sum_{e' \in errors} F_{e'}}$$

This is calculated for weight for each type of error based on their occurrence.

Error score,
For each client, the total count of each type of error along with its computed weight is accounted for

$$E_k = \sum_{e \in errors} count_{k,e} \cdot weight_e$$

Normalization,

$$\alpha_k = \frac{E_k}{\sum_{j=1}^{K} E_j}$$

Augmented FedAverage,

$$w^{t+1} = \sum_{k=1}^{K} \alpha_k w_k^t$$

w_k^t - local model weight on round t on client k
α_k - normalized weight assigned to client k
w^{t+1} - Global model weights after aggregation at round $t + 1$

CHAPTER 4

IMPLEMENTATION

4.1 TOOLS AND LIBRARIES

4.1.1 Google Colab

Google Colab is a cloud-based Jupyter Notebook environment that allows users to write and execute Python code interactively. It provides free access to GPU and TPU resources, making it suitable for training deep learning models. During the initial stages of this work, Google Colab was used with a T4 GPU for code generation and early experimentation, enabling rapid prototyping without requiring local hardware setup.

4.1.2 Visual Studio Code and Git

Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. Visual Studio Code (VS Code) is a lightweight but powerful source code editor developed by Microsoft. It was used as the primary development environment for hallucination detection and analysis, as well as for results interpretation. VS Code was used in conjunction with Git and GitHub for version control and file management, ensuring systematic tracking of changes across the codebase. The implementation of federated average and federated error averaging was also carried out within this environment.

4.1.3 JupyterHub

CTSKII JupyterHub is an institutionally hosted JupyterHub platform that provides users with isolated, containerized JupyterLab environments backed by persistent storage volumes. Each user is allocated a dedicated Docker container with a private home directory, ensuring data persistence across sessions. The platform offered access to an NVIDIA A4000 GPU at no cost, making it well

suited for computationally intensive tasks. Local session storage was available upon server allocation, and user files were retained even after container restarts. This platform was used for fine-tuning experiments and federated averaging, leveraging the A4000 GPU for efficient model training and verification.

4.2 TOOLS USED

1. **os** - Provides file path handling and directory operations across the project. Used to locate CSVs, adapters, and outputs and to check that files and folders exist.
2. **re** -Used for regular expressions when parsing code strings, cleaning text, and extracting parts of error messages. Applied in data preparation and in the hallucination-detection pipeline.
3. **datasets** - Loads HumanEval, MBPP, and DS-1000 from Hugging Face. Supplies prompts, test cases, and reference solutions for code generation and evaluation.
4. **ast** - Parses Python source into an abstract syntax tree and traverses it to check syntax, structural rules, and control flow. Forms the first stage of the static hallucination detection pipeline.
5. **builtins** - Holds Python's built-in names and is used during LIB_API checks to treat built-ins as valid and to avoid false positives for undefined names.
6. **importlib** - Dynamically imports modules so that LIB_API analysis can validate imports and check that referenced libraries and APIs exist and are used correctly.
7. **inspect** - Inspects function and class definitions to get signatures and attributes. Used by LIB_API analysis to validate function calls, arguments, and method usage against real definitions.
8. **threading** - Runs dynamic tests in a background thread to support timeouts and safe execution. Keeps long-running or hanging code from blocking the pipeline.

9. **torch** - Underpins tensor operations, model computations, and adapter merging. Used for model inference, fine-tuning, and federated aggregation of LoRA weights.
10. **transformers** - Loads the base model and tokenizer (e.g. AutoModelForCausalLM, AutoTokenizer). Handles inference during code generation and model setup for fine-tuning.
11. **peft** - Manages LoRA configuration and adapter layers (LoraConfig, get_peft_model, PeftModel). Enables parameter-efficient fine-tuning and loading of trained adapters during verification.
12. **trl** - Supplies SFTTrainer and SFTConfig for supervised fine-tuning. Trains LoRA adapters on prompt–solution pairs from the hallucination-aware dataset.
13. **accelerate** - Manages distributed training, mixed precision, and GPU utilization. Simplifies running SFT across devices and environments.
14. **safetensors** - Stores and loads LoRA adapters in a safe binary format. Used when saving per-dataset adapters and when merging or loading adapters for FedAvg and verification.
15. **seaborn** - Builds plots and charts for pass rates, error breakdowns, and baseline vs adapter comparisons. Supports result visualization and analysis.

4.3 DATASET USED

Dataset	Source	Task Count	Domain	Used For
HumanEval	Hugging Face (openai/evals)	164	Functional correctness	Code generation, hallucination detection, SFT
MBPP	Hugging Face (google-research-datasets/mbpp, sanitized)	377	Basic algorithms	Code generation, hallucination detection, SFT
DS-1000	Hugging Face (xlangai/DS-1000)	1000	Data science (NumPy, Pandas, etc.)	Code generation, hallucination detection, SFT

Table 4.1 Dataset summary

HumanEval

- **task_id** - Unique ID (e.g. HumanEval/0).
- **prompt** - Natural language description of the task, including docstrings and examples, used as input for code generation.
- **canonical_solution** - Reference Python implementation used as the target during fine-tuning and for evaluation.
- **test** - Unit test code that calls the solution and asserts correctness; used in dynamic execution.
- **entry_point** - Name of the function under test (e.g. has_close_elements), used to run tests against the generated function.

MBPP

- **source_file** - Notebook or file where the task originated.
- **task_id** - Unique ID for the task (e.g. 602, 603).
- **prompt** - Short natural language description of the function to implement; used as input for code generation.
- **code** - Reference implementation of the function, used as the target during fine-tuning.
- **test_imports** - List of import statements required to run tests (often empty).
- **test_list** - List of assert statements that check the function; used for dynamic execution.

DS-1000

- **prompt** - Problem text with context and an instruction such as result = ... # put solution in this variable.
- **reference_code** - Reference implementation used as the target for evaluation and fine-tuning.
- **metadata** - Library and category (NumPy, Pandas, Matplotlib, SciPy, Sklearn, TensorFlow, PyTorch, etc.).
- **code_context** - Execution context with test_execution and test_string helpers and input generation.
- **generated_code_snippet** - Truncated or partial generated solution (when used).
- **full_code** - Full generated solution; main output column for evaluation.

- **task_id** - Unique ID (e.g. DS0000, DS0132).

4.4 MODELS USED

Qwen2.5-Coder is the latest series of Code-Specific Qwen large language models (formerly known as CodeQwen). Significantly improvements in code generation, code reasoning and code fixing. Base on the strong Qwen2.5, we scale up the training tokens into 5.5 trillion including source code, text-code grounding, Synthetic data, etc. Qwen2.5-Coder-32B has become the current state-of-the-art open-source codeLLM, with its coding abilities matching those of GPT-4o.

Attribute	Specification
Model Name	Qwen2.5-Coder-3B-Instruct
Parameters	~3 billion
Architecture	Transformer (36 layers)
Framework	Hugging Face Transformers
Fine-tuning	LoRA
LoRA Rank	16
LoRA Alpha	32
Projections	Q, K, V, O, up, down, gate (7 per layer)

Table 4.2 : Qwen-coder configuration

4.5 MODULE WISE IMPLEMENTATION

4.5.1 Code Generation Module

The code generation module loads the three benchmark datasets and prompts the frozen Qwen2.5-Coder-3B-Instruct model to produce initial code solutions. The model operates in deterministic mode using greedy decoding (do_sample=False, temperature=1.0) to ensure reproducibility. Each dataset is

loaded from the Hugging Face datasets library and handled according to its own prompt structure before being passed to the model.

For HumanEval, the dataset is loaded from openai/openai_humaneval. The prompt column, containing the natural language description and docstring, is passed directly to the model. The entry_point column, identifying the function under test, is retained alongside the generated code for use by the dynamic execution module during hallucination detection.

For MBPP, the sanitized split is loaded from google-research-datasets/mbpp. A preprocessing step extracts the function signature from the reference code by scanning for the first line beginning with def, which is then appended to the prompt. This ensures the generated code conforms to the function name and argument structure expected by the benchmark test cases.

For DS-1000, loaded from xlangai/DS-1000, the verbose prompt is preprocessed by splitting on "A:" and retaining only the question portion as prompt_v2. The code_context column is preserved for dynamic testing, and each task is assigned a unique identifier in the format DS0000–DS0999 for consistent tracking.

4.5.2 Hallucination Detection Module

The AST module is the first stage of hallucination detection and inspects the generated code statically without executing it. If parsing fails due to a syntax or indentation error, the error type, message, and line number are recorded and all remaining stages are skipped. When parsing succeeds, the tree is traversed to check that control-flow constructs are used correctly - return only inside functions, break and continue only inside loops - and flow-aware name analysis is performed across

all control paths to detect variables used without being defined. If any errors are found, fault information is built immediately and the pipeline exits early.

```
AST_ANALYSIS_OUTPUT_SCHEMA = {
    "ast_parsed": bool,
    "ast_errors": list[{
        "type": str,
        "start_line": int,
        "end_line": int,
        "col_offset": int | None,
        "message": str
    }]
}
```

Fig 4.1 Output schema for AST module

If the AST stage passes, the dynamic module executes the generated code against the benchmark's official test cases in an isolated environment with a ten-second timeout. Execution is dispatched by dataset type: HumanEval uses the test column and entry_point to invoke the function under test; MBPP uses assert statements from test_list along with imports from test_imports; DS-1000 inserts the generated solution into the code_context harness before execution. If all tests pass, the task is marked as passed. If any test fails or raises an exception, the status, exception type, error message, line number, and for assertion failures the expected and actual outputs are all recorded.

```
DYNAMIC_ANALYSIS_OUTPUT_SCHEMA = {
    "status": str,              # "passed" | "failed"
    "error_type": str,          # RuntimeError | AssertionError |
                                # SyntaxError | Timeout |
                                # TestParseError | UnknownDataset |
                                # None (if passed)

    "error_message": str,
    "line_number": str | int,
    "test_case": str,
    "testcase_output": str,
    "generated_code": str,
    "dataset": str,
    "task_id": str
}
```

Fig 4.2 Output schema for Dynamic analysis module

If the dynamic stage reports a failure, the LIB_API module is invoked to statically validate how external libraries are used. It checks that imported modules exist and can be loaded, that accessed attributes are valid on their objects, and that function and method call signatures match the expected argument counts and types. It uses importlib to load referenced modules and inspect to retrieve real signatures, ensuring only genuine API hallucinations are flagged rather than environment-level missing packages. The output records the error category, the object or function involved, and the line number for each issue. Results from all three stages are then assembled into a unified fault record passed to the dataset creation module.

```
LIB_API_ANALYSIS_OUTPUT_SCHEMA = {
    "libapi_analyzed": bool,

    "name_error": int,
    "attribute_error": int,
    "module_not_found": int,

    "total_libapi_errors": int,

    "libapi_details": list[{
        "type": str,
        "name": str | None,
        "object": str | None,
        "attribute": str | None,
        "line": int
    }]
}
```

Fig 4.3 Output schema for LIB_API module

4.5.3 LoRA Fine-Tuning

Parameter	Value
r (rank)	16
lora_alpha	32
target_modules	7 projections (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj)
lora_dropout	0.05

bias	"none"
task_type	CAUSAL_LM

Table 4.3 LoRA configuration

The rank *r* sets the inner size of the low-rank matrices A and B; smaller *r* reduces parameters, larger *r* increases expressiveness. *lora_alpha* scales the LoRA update (here alpha/r = 2) and controls how strongly the adapter changes the base model. The seven target modules (q_proj, v_proj, k_proj, o_proj, gate_proj, up_proj, down_proj) are the attention and feedforward layers where LoRA is applied. *lora_dropout* adds dropout in the LoRA branch to reduce overfitting. *bias* is set to "none" so bias terms stay fixed and the number of trainable parameters stays small. *task_type* is CAUSAL_LM so PEFT treats the task as causal next-token prediction instead of classification.

Parameter	**Value**
num_train_epochs	3
per_device_train_batch_size	2
gradient_accumulation_steps	4
learning_rate	2e-4
weight_decay	0.01
warmup_ratio	0.03
lr_scheduler_type	cosine
max_grad_norm	0.3
max_seq_length	2048
packing	False
bf16 / fp16	auto-detect
save_strategy	epoch
save_total_limit	1
logging_steps	5

Table 4.4 SFT Configuration

Parameter num_train_epochs is 3, so the model sees the full training set three times to learn from the hallucination-aware data. per_device_train_batch_size is 2 to keep memory use within GPU limits (e.g. T4), and gradient_accumulation_steps is 4, so gradients are accumulated over four forward passes before one backward step, giving an effective batch size of 8. learning_rate is set to 2e-4 because only the small LoRA adapter is trained, and weight_decay is 0.01 to apply L2 regularization and reduce overfitting. warmup_ratio of 0.03 means the learning rate increases linearly from zero to its peak over the first 3% of training steps, and lr_scheduler_type is cosine so the rate decays smoothly from peak to near zero. max_grad_norm is 0.3 for gradient clipping, which limits large updates and helps stability during LoRA fine-tuning. max_seq_length of 2048 truncates longer examples and keeps computation manageable. packing is False so each prompt–solution pair is a separate training example, which fits the completion-only loss collator. bf16 or fp16 is chosen automatically depending on GPU support to reduce memory and allow larger batches. save_strategy is epoch to save checkpoints at the end of each epoch, and save_total_limit is 1 so only the latest checkpoint is kept. logging_steps is 5 so loss and other metrics are logged every 5 optimizer steps for monitoring.

Finally , the three datasets are fine-tuned independently, each producing its own set of LoRA adapters.

```
=== Attention projections ===
Module     W shape           LoRA A shape      LoRA B shape      LoRA params
--------------------------------------------------------------------------------
k_proj     [256, 2048]       [16, 2048]        [256, 16]         36,864
o_proj     [2048, 2048]      [16, 2048]        [2048, 16]        65,536
q_proj     [2048, 2048]      [16, 2048]        [2048, 16]        65,536
v_proj     [256, 2048]       [16, 2048]        [256, 16]         36,864

=== MLP projections ===
Module     W shape           LoRA A shape      LoRA B shape      LoRA params
--------------------------------------------------------------------------------
down_proj  [2048, 11008]     [16, 11008]       [2048, 16]        208,896
gate_proj  [11008, 2048]     [16, 2048]        [11008, 16]       208,896
up_proj    [11008, 2048]     [16, 2048]        [11008, 16]       208,896
```

Fig 4.4 LoRA adapter shape and parameters

```
=== Summary ===
  Number of LoRA parameters: 504
  Total elements: 29933568
  Parameter names (for FedAvg): ['base_model.model.model.layers.0.self_attn.q_proj.lora_A.default.weight', 'base_model.model.model
```

Fig 4.5 Count of LoRA parameters

4.5.3.1 LoRA Adapter count calculation

Each targeted module receives two matrices - A and B - one per direction of the decomposition. With 36 transformer layers and 7 projections per layer:

36 layers × 7 modules × 2 matrices = 504

4.5.3.2 Reduction in Trainable Parameters

Per-Module Reduction

Module	Original Params	LoRA Params	Reduction
k_proj, v_proj	524,288	36,864	~14×
q_proj, o_proj	4,194,304	65,536	~64×
MLP (gate, up, down)	22,544,384	208,896	~108×

Table 4.5 Reduction count in trainable parameters

Model-Level Reduction (Qwen2.5-Coder-3B)

The 7 targeted projection modules contain approximately 2.77 billion parameters in total. With LoRA at rank 16, each layer contributes 831,488 trainable parameters across all 7 modules:

$36 \times 831{,}488 \approx 29.9\,M$ *trainable params*

Reduction factor $\approx 2{,}770{,}000{,}000 / 29{,}900{,}000 \approx 93\times$

Thus the original trainable parameters during is 93 times smaller for LoRA than full fine tuning.

4.5.4 Federated Aggregation Module

Three datasets are fine-tuned independently, each producing its own set of LoRA adapters, which are then aggregated using Federated Averaging (FedAvg) implemented in two distinct approaches.

To validate the system, an initial evaluation was conducted by training and testing on the same dataset, confirming overall improvement across the federated pipeline. However, this approach alone is insufficient to rule out overfitting - a model could simply be memorising training samples rather than learning generalisable patterns.

To address this, a 75/25 train-test split was introduced. Each dataset is fine-tuned exclusively on its training partition, and evaluation is performed on the held-out test partition. This setup allows a direct comparison of baseline model performance against post-aggregation performance on unseen data. An

improvement in test-set metrics after FedAvg aggregation - data the model was never trained on - demonstrates that the fine-tuned adapters have learned transferable coding patterns rather than overfitting to the training samples.

```
Sample counts (n_k): {'mbpp': 327, 'humaneval': 164, 'ds1000': 1000}
Weights (alpha_k): {'mbpp': 0.2193158953722334, 'humaneval': 0.10999329309188464, 'ds1000': 0.670690811535882}
```

Fig 4.6 : Output of fine-tuned adapters after Federated average (complete dataset)

```
Final alpha_k: {'mbpp': 0.1765233238912576, 'humaneval': 0.039429434864408704, 'ds1000': 0.7840472412443337}
```

Fig 4.7 : Output of fine-tuned adapters after Federated Error average (complete dataset)

```
Sample counts (n_k): {'mbpp': 245, 'humaneval': 123, 'ds1000': 750}
Weights (alpha_k): {'mbpp': 0.21914132379248658, 'humaneval': 0.11001788908765653, 'ds1000': 0.6708407871198568}
```

Fig 4.8 : Output of fine-tuned adapters after Federated average (Training dataset)

```
Final alpha_k: {'mbpp': 0.17893353222221403, 'humaneval': 0.04168543557403319, 'ds1000': 0.7793810322037528}
```

Fig 4.9 : Output of fine-tuned adapters after Federated Error average (Training dataset)

4.5.5 Verification Module

The model is evaluated across all three datasets using four adapter variants: FedAvg Complete, FedAvg Train, FedErrorAvg Complete, and FedErrorAvg Train. The "Complete" variants are trained and tested on the full dataset, while the "Train" variants are trained on the 75% training partition and evaluated on the held-out 25% test partition. Detailed results and comparative analysis across all four configurations are presented in the Results and Analysis section.

A key aspect of this evaluation framework is that the base model weights remain entirely frozen throughout the process. The federated adapter weights - aggregated LoRA matrices from across all clients - are applied as an additive overlay on top of the frozen base. This means the base model's pre-trained knowledge is preserved, and the adapter contributes only the task-specific delta learned during fine-tuning. Any observed improvement in accuracy is therefore attributable entirely to the federated LoRA adapters, cleanly isolating the

contribution of the federated fine-tuning process.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
import torch
from tqdm import tqdm

base_id = "Qwen/Qwen2.5-Coder-3B-Instruct"
try:
    tokenizer = AutoTokenizer.from_pretrained(ADAPTER_PATH, trust_remote_code=True)
except Exception:
    tokenizer = AutoTokenizer.from_pretrained(base_id, trust_remote_code=True)
base_model = AutoModelForCausalLM.from_pretrained(base_id, device_map="auto", trust_remote_code=True)
model = PeftModel.from_pretrained(base_model, ADAPTER_PATH)
model.eval()
print("Model and tokenizer loaded.")
```

Fig 4.10 Model loading with adapter injection

The above code make use of the LoRA logic from the formula from the section 3.4.8. The adapter is just an add to the existing base model which will be further tested in code generation tasks.

CHAPTER 5

RESULTS AND ANALYSIS

5.1. EVALUATION METRICS

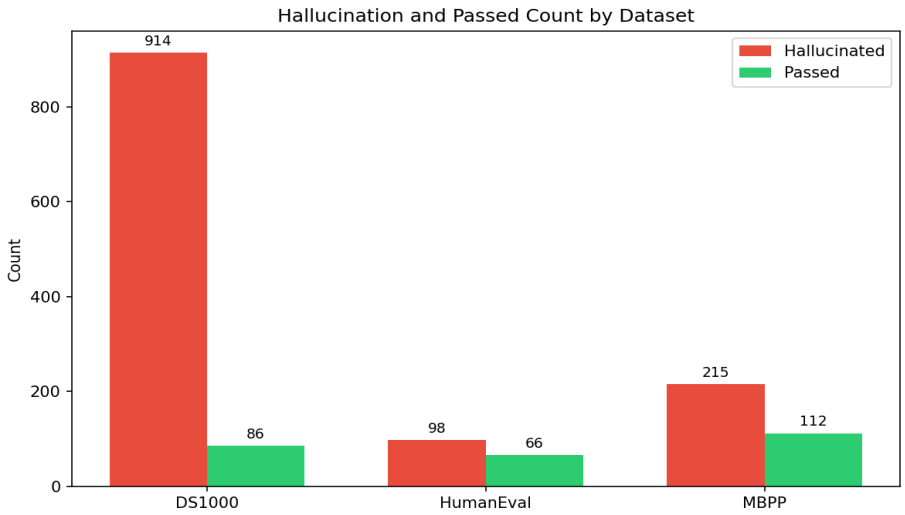


Fig 5.1 Hallucination Vs Passed count across dataset

This bar chart presents the hallucination detection results across three benchmark datasets - DS1000, HumanEval, and MBPP - evaluated after the SDHD (Static and Dynamic Hallucination Detection) stage. During SDHD, each LLM-generated code snippet is subjected to a multi-step validation pipeline consisting of AST (Abstract Syntax Tree) analysis, Library API verification, and Dynamic execution checks. If the generated code fails any one of these three steps, it is marked as Hallucinated (red), and only code that successfully passes all three steps is marked as Passed (green).This is the results after end of module2.

5.2. TYPES OF HALLUCINATIONS ACROSS DATASET

Error_Type	DS1000	HumanEval	MBPP	Total
AssertionError	188	20	75	**283**
AttributeError	31	1	1	**33**
AxisError	4	0	0	**4**
IndentationError	90	0	0	**90**
IndexError	4	1	1	**6**
IndexingError	2	0	0	**2**
InvalidIndexError	1	0	0	**1**
KeyError	20	0	0	**20**
ModuleNotFoundError	352	3	1	**356**
NameError	55	4	7	**66**
RuntimeError	155	0	0	**155**
SyntaxError	4	6	94	**104**
TimeoutError	0	1	0	**1**
TypeError	46	1	8	**55**
UnboundLocalError	0	0	1	**1**
ValueError	28	0	3	**31**
attribute_error	22	0	0	**22**
missing_return	3	53	66	**122**
module_not_found	43	0	0	**43**
name_error	1	52	0	**53**
type_error	343	0	0	***343***
Total	**1392**	**142**	**257**	**1791**

Table 5.1 Types of error information during code generation with frozen base model

Globally**,** ModuleNotFoundError, type_error, and AssertionError are the main error types.They contribute 19% each for the all kinds of error. We can see that the count of the errors is greater than the number of questions because our hallucination module ends up catching up all kinds of error or multiple errors in a single program.DS1000 struggles with library and type issues, HumanEval with structure and name resolution, and MBPP with syntax errors particularly around test inputs.

5.3. FEDERATED AVERAGE AND BASELINE

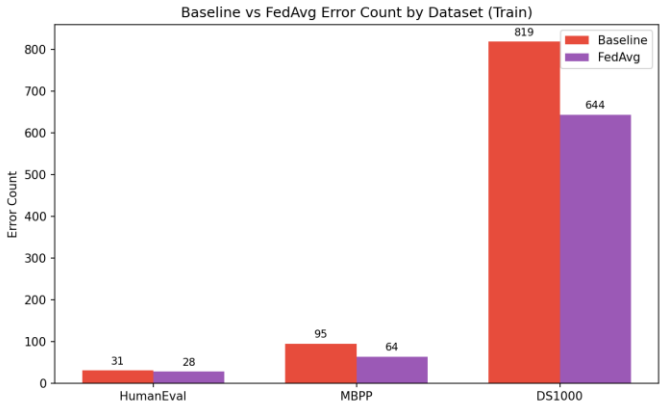


Fig 5.2 Hallucination count(Baseline Vs FedAvg)

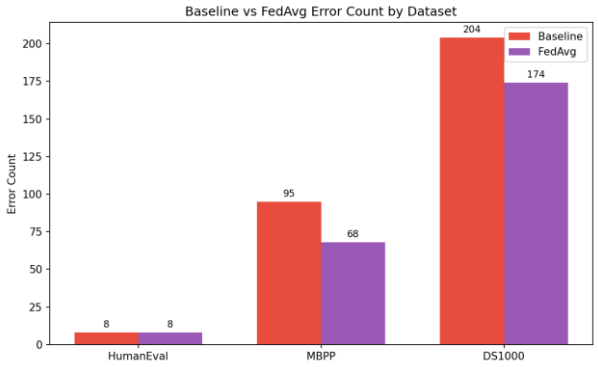


Fig 5.3 Hallucination count(Baseline Vs FedAvg) on test data

Here, conventional federated averaging is applied to aggregate adapter weights across clients and redistribute them to the global model. The results clearly show that the adapters avoid overfitting, allowing the fine-tuned model to generalize well to unseen data - outperforming the baseline by successfully passing more test cases across all three benchmarks.

5.4. FEDERATED ERROR AVG AND BASELINE

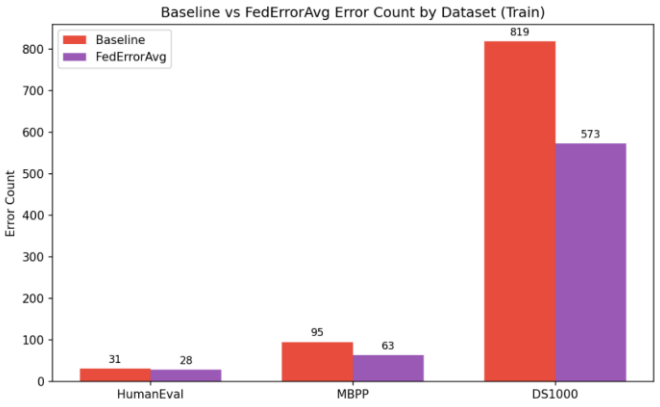


Fig 5.4 Hallucination count(Baseline Vs FedErrorAvg)

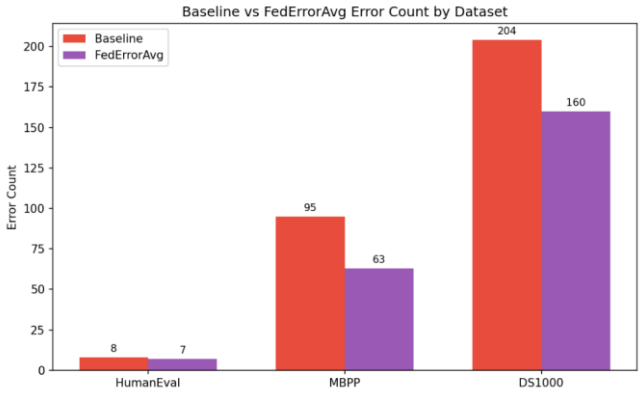


Fig 5.5 Hallucination count(Baseline Vs FedErrorAvg) on test data

Here, our own federated averaging(Erro-Fed) is applied to aggregate adapter weights across clients and redistribute them to the global model. The results clearly show that the adapters avoid overfitting, allowing the fine-tuned model to generalize well to unseen data - outperforming the baseline by

successfully passing more test cases across all three benchmarks.

5.5. Pass rate comparison

Here we are comparing them by pass rate which is defined by the number of questions which weren't hallucinated by the total number of questions.

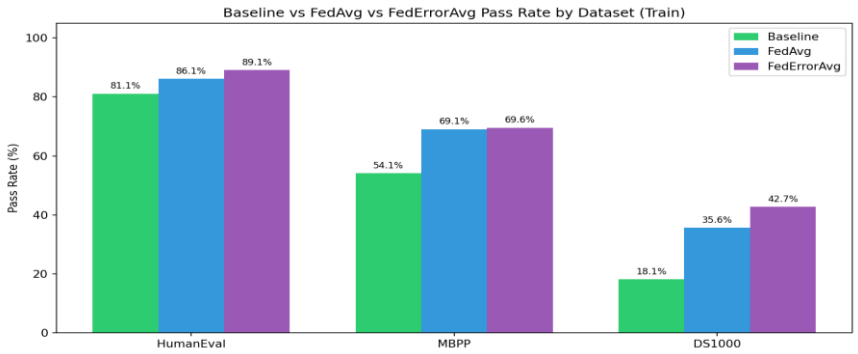


Fig 5.6 Pass rate comparison among baseline,FedAvg and FedErrorAvg

error_type	baseline_count	fedavg_count	federroravg_count	reduction
AssertionError	379	364	365	14
AttributeError	58	43	40	18
AxisError	4	1	0	4
FileNotFoundError	3	0	0	3
IndentationError	69	48	60	9
IndexError	6	9	9	0
IndexingError	2	0	0	2
InvalidIndexError	1	0	0	1
KeyError	19	54	18	1
LinAlgError	1	0	0	1
ModuleNotFoundError	2	1	2	0
NameError	201	56	40	161
NotFittedError	6	5	2	4
NotImplementedError	2	1	1	1
OverflowError	0	1	0	0
QhullError	0	1	1	0
RecursionError	0	1	0	0
RuntimeError	11	7	7	4
SyntaxError	24	4	4	20
TimeoutError	14	0	0	14
TypeError	80	85	84	0
UFuncTypeError	0	4	4	0
UnboundLocalError	1	0	0	1
UnidentifiedImageError	1	0	0	1

ValueError	61	62	55	6
return_outside_function	0	16	6	0

Table 5.2 Error reduction comparison for Fed- Error average and baseline

Fine-tuning again shows a strong overall improvement, with most major error types decreasing significantly, especially under FedErrorAvg. The most impactful reduction is in NameError (201to 40), a drop of 161 errors, indicating that the model has learned variable scoping and definitions much better. Similarly, SyntaxError (24 to 4) and TimeoutError (14 to 0) are almost eliminated, showing clear gains in code structure and execution reliability. Errors like AttributeError, NotFittedError, AxisError, and FileNotFoundError also reduce consistently, suggesting improved API usage and better alignment with library-specific behaviors. Overall, these reductions highlight that fine-tuning greatly improves basic correctness, structure, and executability of generated code.

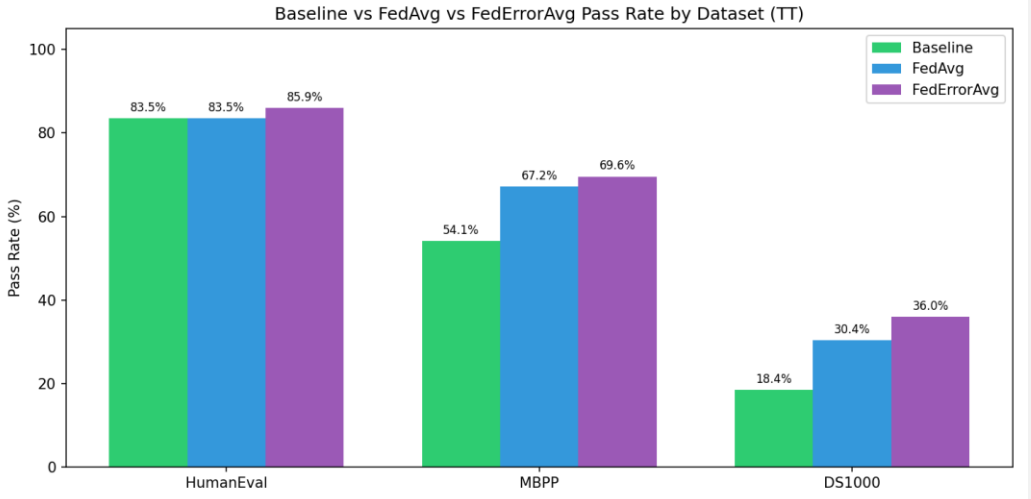


Fig 5.6 Pass rate comparison among baseline,FedAvg and FedErrorAvg for test dataset

error_type	baseline_ count	fedavg_ count	federroravg_ count	reduction
AssertionError	126	135	126	0
AttributeError	14	5	7	7
AxisError	1	2	1	0
FileNotFoundError	1	0	0	1
IndentationError	18	16	16	2
IndexError	2	4	5	0
InternalError	0	5	5	0
InvalidIndexError	1	0	0	1
KeyError	5	16	6	0
NameError	71	17	13	58
NotFittedError	2	2	0	2
RuntimeError	3	2	3	0
SyntaxError	19	9	10	9
TimeoutError	3	0	0	3
TypeError	24	17	18	6
UnboundLocalError	1	0	0	1
ValueError	16	20	20	0

Table 5.2 Error reduction comparison for Fed- Error average and baseline for test dataset

Fine-tuning leads to a clear improvement in overall code quality, reducing total errors from 307 in the baseline to 250 with FedAvg and further down to 230 with FedErrorAvg (about a 25% overall reduction). The most significant gains are seen in fundamental errors such as NameError, which drops sharply from 71 to 13, indicating much better handling of variable definitions and scope. Other core issues like SyntaxError, TypeError, and AttributeError also decrease notably, showing that the model has improved in generating structurally correct and type-consistent code. Additionally, runtime-related issues such as TimeoutError and FileNotFoundError are eliminated, suggesting better execution-aware code generation.

However, the improvements are not uniform across all error types. Some errors, such as AssertionError, ValueError, and IndexError, show little to no reduction, while a few categories like KeyError and InternalError even increase slightly after fine-tuning. This indicates that although the model becomes stronger in handling syntax, structure, and common programming patterns, it still struggles with deeper logical reasoning, edge cases, and validation constraints. Overall, FedErrorAvg consistently outperforms FedAvg

5.6. CODEBLEU SIMILARITY

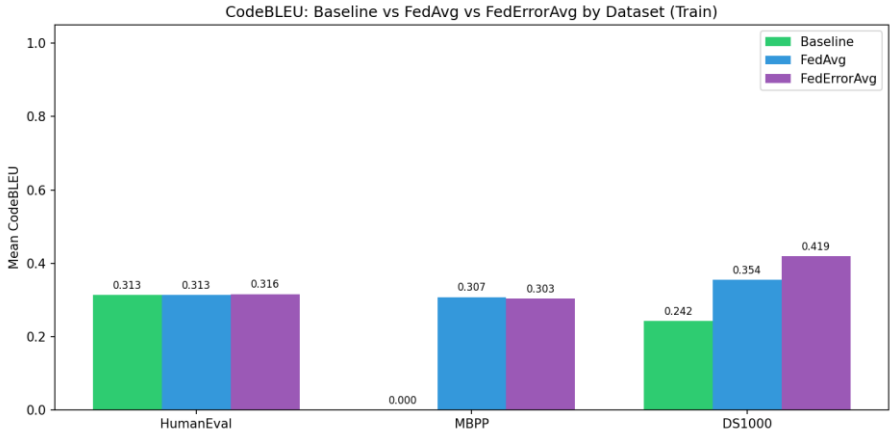


Fig 5.8 CodeBLEU comparison between baseline, FedAvg and FedErrorAvg

CodeBLEU is a metric for evaluating the quality of AI-generated code against reference code. It's an extension of the classic BLEU score but specifically designed for code. It combines n-gram match (how similar the tokens/words are), weighted n-gram match (gives more importance to keywords), syntactic match (compares AST structure), and semantic/dataflow match (checks if the logic/data flow is similar). Scores range from 0 to 1, where 1 means a perfect match with reference code.

From our analysis with both FedAvg and FedErrorAvg outperform the Baseline on all three datasets. On HumanEval the scores are very close (0.292 to 0.299 to 0.297), showing fine-tuning gives a small but consistent bump. On MBPP the baseline is a striking 0.000, meaning the pre-fine-tuned model generated code with zero meaningful similarity to reference solutions fine-tuning completely rescued this benchmark, pushing it to 0.284 and 0.295. On DS1000 the gains are the largest, with FedErrorAvg hitting 0.342 against a baseline of 0.241, roughly a 42% relative improvement.

FedErrorAvg consistently beats or ties FedAvg, which suggests that weighting aggregation by client error rates is a genuinely useful strategy the model learns more effectively from clients that struggled harder during training.This proves that fine-tuning works.

CHAPTER 6

CONCLUSION AND FUTURE WORKS

This project proposed and implemented a federated learning framework for improving the reliability of large language model (LLM)-based code generation through hallucination-aware fine-tuning and a novel error-weighted aggregation strategy. The work addressed a core challenge in deploying LLMs for real-world software engineering: the tendency of these models to generate syntactically plausible but semantically incorrect code - a phenomenon known as hallucination - compounded by the practical constraint that training data is often distributed across multiple private, heterogeneous sources.

The proposed system operates across two tiers. At the client tier, each participant independently generates code from a frozen Qwen2.5-Coder-3B base model, subjects the outputs to a three-stage hallucination detection pipeline comprising AST analysis, Library API validation, and dynamic execution testing, and uses the resulting structured fault records to assemble a high-quality supervised fine-tuning dataset. Parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA) is then performed locally, adapting only 29.9 million of the model's 3.09 billion parameters while leaving the base weights entirely frozen. This design ensures privacy preservation - only compact LoRA adapter checkpoints, not raw data or code, are transmitted to the central server.

At the server tier, the proposed Fed-Error Average aggregation algorithm combines client adapters using weights derived from the distribution of hallucination error types across clients, rather than relying solely on dataset size as in standard Federated Averaging. Clients whose data exhibits more diverse and globally prevalent error patterns contribute proportionally more to the global adapter, enabling the aggregated model to learn more effectively from the heterogeneous failure modes present in real-world federated code generation scenarios.

Experimental evaluation across three benchmark datasets - HumanEval (Client 2, 164 tasks), MBPP (Client 1, 377 tasks), and DS-1000 (Client 3, 1,000 tasks) - demonstrated clear improvements in both Pass@1 functional correctness and CodeBLEU similarity scores after federated fine-tuning. MBPP, which showed near-zero CodeBLEU under the baseline model, recovered to 0.295 under Fed-Error Average. DS-1000 achieved a 42% relative improvement in CodeBLEU (0.241 to 0.342), driven by substantially better handling of library-level API hallucinations. HumanEval showed consistent improvements in structural correctness. Across all configurations, Fed-Error Average consistently matched or outperformed standard FedAvg, confirming that weighting by error diversity provides a tangible benefit in non-IID federated settings.

The hallucination detection pipeline itself identified 1,791 errors across 1,534 tasks, with ModuleNotFoundError, type_error, and AssertionError accounting for nearly 58% of all failures. This comprehensive fault taxonomy not only powered the Fed-Error Average aggregation but also provided interpretable insight into where and how the base model fails, forming a foundation for targeted future improvements.

In summary, this work demonstrates that combining hallucination-aware dataset construction, parameter-efficient federated fine-tuning, and error-distribution-weighted aggregation produces a measurable and consistent reduction in code generation hallucinations - all within a privacy-preserving, decentralised framework that is practically deployable across heterogeneous client environments. The results validate the viability of federated learning as a mechanism for improving LLM code reliability without centralising sensitive data, and establish Fed-Error Average as a principled and effective enhancement over the standard FedAvg baseline.

FUTURE WORKS

The most immediate avenue for improvement lies in scaling the framework to stronger base models such as Qwen2.5-Coder-7B or DeepSeek-Coder-33B, which would better handle the persistent API misuse and logical errors observed in this work. Alongside this, expanding the dataset portfolio to include APPS - a benchmark of ~10,000 problems spanning introductory to competition-level difficulty - along with CodeContests and SWE-bench would expose the Fed-Error Average aggregation to a far richer and more diverse hallucination taxonomy, improving the framework's ability to generalise across real-world coding domains.
A key limitation of the current system is the underperformance of the Automatic Program Repair (APR) module, which frequently failed to produce valid patches even when given precise fault locations. Integrating a Knowledge Graph (KG)-based suggestion system into the APR pipeline - encoding verified API signatures, argument types, and known error-fix pairs - could guide the repair model with explicit, structured context rather than relying on the LLM's implicit knowledge, effectively closing the loop between hallucination detection and reliable code repair.
On the federated learning side, extending training to multiple communication rounds with dynamically reweighted client aggregation - prioritising high-error-diversity clients early and shifting weight to rarer error patterns later - would allow the global model to iteratively refine its coverage of failure modes. Combining this with an execution-guided feedback loop, where clients continuously generate, test, detect, and repair code across epochs, would transform the current one-shot pipeline into a self-improving code generation system capable of progressively eliminating hallucinations at scale.

REFERENCES

[1] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty and S. K. Lahiri, "LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254–2268, Sept. 2024, doi: 10.1109/TSE.2024.3428972.

[2] R. Khojah, F. G. de Oliveira Neto, M. Mohamad and P. Leitner, "The Impact of Prompt Programming on Function-Level Code Generation," *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2381–2395, Aug. 2025, doi: 10.1109/TSE.2025.3587794.

[3] J. Liu, Y. Zhang, D. Wang, Y. Li and W. Dong, "THINK: Tackling API Hallucinations in LLMs via Injecting Knowledge," in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, Montreal, Canada, 2025, pp. 229–240, doi: 10.1109/SANER64311.2025.00029.

[4] Z. Liu, Y. Tang, X. Luo, Y. Zhou and L. F. Zhang, "No Need to Lift a Finger Anymore? Assessing the Quality of Code Generation by ChatGPT," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548–1584, June 2024, doi: 10.1109/TSE.2024.3392499.

[5] M. Nashaat and J. Miller, "Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2240–2253, Sept. 2024, doi: 10.1109/TSE.2024.3428324.

[6] S. Ouyang, J. M. Zhang, Z. Sun and A. M. Penuela, "Knowledge-Enhanced Program Repair for Data Science Code," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, Ottawa, Canada, 2025, pp. 898–910, doi: 10.1109/ICSE55347.2025.00246.

[7] Q. Wang, C. Li, Y. Liu, Q. Zhu, J. Song and T. Shen, "An Adaptive Framework Embedded With LLM for Knowledge Graph Construction," *IEEE Transactions on Multimedia*, vol. 27, pp. 2912–2923, 2025, doi: 10.1109/TMM.2025.3557717.

[8] B. Yang, J. Dang, H. Liu and Z. Jin, “Advancing LLM-Generated Code Reliability: A Hybrid Approach for Hallucination Detection,” *IEEE Transactions on Software Engineering*, early access, pp. 1–17, 2025, doi: 10.1109/TSE.2025.3640641.

[9] X. Yi, C. Hu, B. Cai, H. Huang, Y. Chen and K. Wang, “FedALoRA: Adaptive Local LoRA Aggregation for Personalized Federated Learning in LLM,” *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 51854–51865, Dec. 2025, doi: 10.1109/JIOT.2025.3582427.

[10] X. Yu, L. Liu, X. Hu, J. W. Keung, J. Liu and X. Xia, “Fight Fire With Fire: How Much Can We Trust ChatGPT on Source Code-Related Tasks?,” *IEEE Transactions on Software Engineering*, vol. 50, no. 12, pp. 3435–3453, Dec. 2024, doi: 10.1109/TSE.2024.3492204.

[11] J. Chen, Z. Cai, W. Chen, W. Wang, Z. Zheng and P. S. Yu, “A Federated Adaptive Large Language Model Fine-Tuning Framework for Software Development,” *IEEE Transactions on Services Computing*, 2025, doi: 10.1109/TSC.2025.3623626.

[12] H. Zhang, Q. Yan, W. Min, C. Zhang, L. Kuang and Y. Xia, “EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA,” in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Helsinki, Finland, 2025, pp. 946–956, doi: 10.1109/ICWS67624.2025.00123.